Heaven's light is our guide

Rajshahi University of Engineering &Technology

Department of Computer Science & Engineering



Course Code : CSE 3206

Lab Report No :6

Submitted by:	**Submitted to:**
Name: Md.Rofaz Hasan Rafiu **Roll(s)** : 2203036 **Section** : A **Session** : 2022-2023	**Name:** Emrana Kabir Hashi **Designation:** Assistant Professor **Department:** CSE **Institution:** RUET

1. Requirements Engineering & Process Model

1.1 Project Overview: CareFlow is a modular, extensible clinic management and billing application developed in Python. It encapsulates five core functional modules: (1) User Authentication & Roles (Doctor, Patient, Staff); (2) Appointment Scheduling with doctor slot exclusivity; (3) Dynamic Invoicing applying healthcare subsidies; (4) Electronic Prescriptions; and (5) Event-Driven Notifications. All data is persistently stored in a relational SQLite database (careflow.db).

1.2 Functional & Non-Functional Requirements:

- **FR-1 (Authentication & RBAC):** Secure registration and authentication for Doctors, Patients, and Staff with salted SHA-256 password hashing.
- **FR-2 (Appointment Management):** Patients book appointments for specific slots. The system detects and blocks conflicting bookings for the same doctor.
- **FR-3 (Polymorphic Billing):** Invoices are dynamically calculated based on selected schemes: Standard (5% VAT), RUET Student (30% waiver), Insurance (80% coverage), or Emergency (25% surcharge).
- **FR-4 (Payment Gateway Adapter):** Unified interface accepts payments via simulated digital gateways such as bKash and Stripe.
- **FR-5 (Electronic Prescriptions):** Doctors record clinical diagnoses and medication instructions, automatically completing confirmed appointments.
- **NFR-1 (Thread Safety & Persistence):** SQLite connection is managed through a thread-safe Singleton to prevent file collisions.
- **NFR-2 (Testability):** Services are decoupled from concrete database and payment APIs using mocks, achieving >85% test coverage.

1.3 User Stories & Acceptance Criteria:

- **User Story 1:** *As a patient, I want to book an appointment with a doctor for a specific date and time, so that I can receive timely medical consultation.*
  Acceptance Criteria: System creates a PENDING appointment; if the doctor is already booked at that slot, the booking is rejected with an error; observers send confirmation emails.
- **User Story 2:** *As a RUET student, I want a 30% medical waiver applied to my clinic bill, so that healthcare remains affordable.*
  Acceptance Criteria: Applying the Student subsidy strategy reduces the consultation fee by 30% with 0% tax; the invoice is marked PAID upon bKash payment.

1.4 Chosen Process Model & Justification:

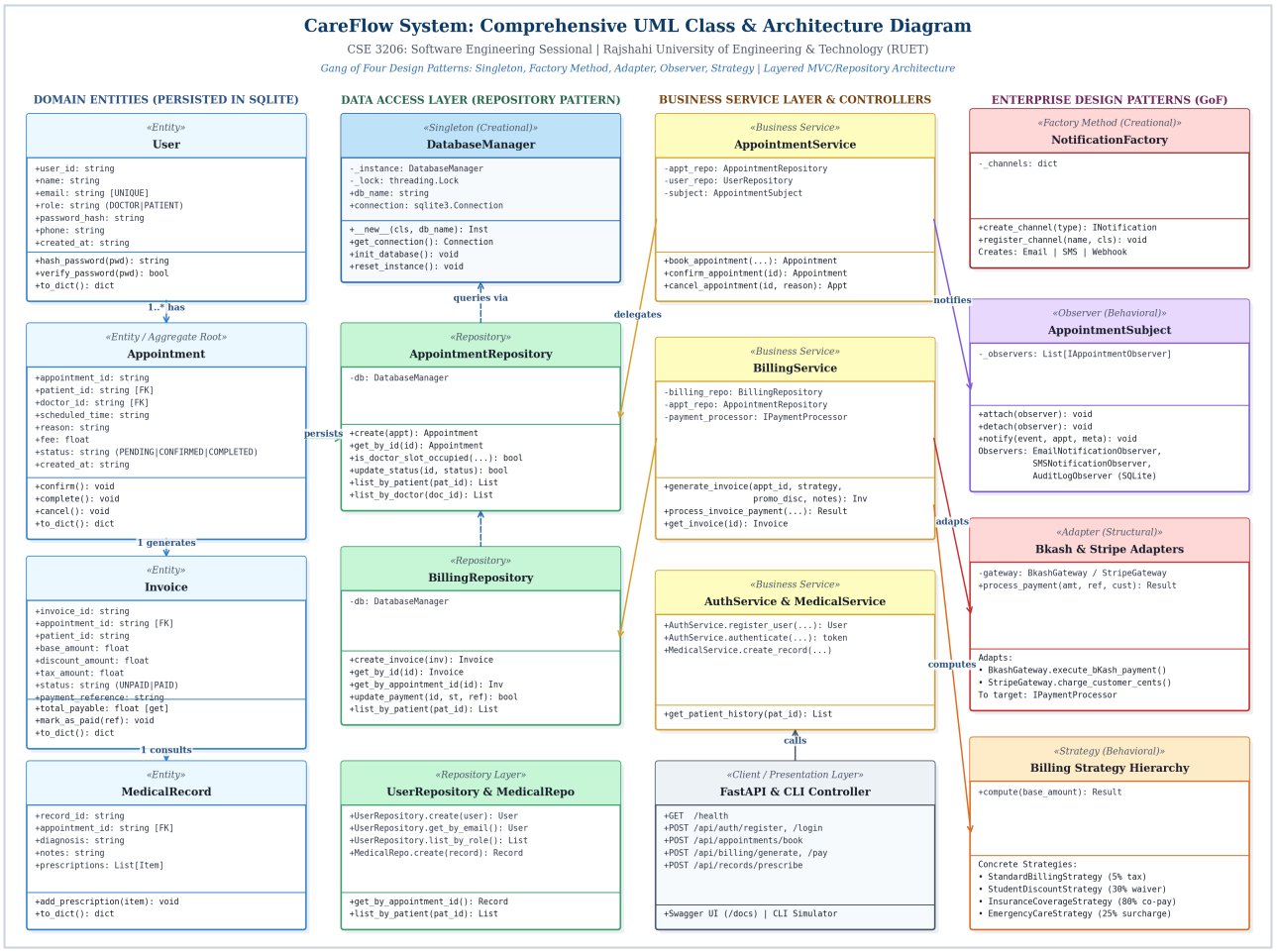
CareFlow was developed using the **Agile Scrum methodology** in four weekly sprints (Sprint 1: Core Models & Singleton DB; Sprint 2: Design Patterns; Sprint 3: Repositories, Services & TDD Mocks; Sprint 4: FastAPI OpenAPI docs & CI/CD). An Agile process was chosen because healthcare requirements—such as doctor scheduling rules and payment gateway parameters—evolve through continuous testing. A rigid Waterfall model was rejected because it delays integration testing and cannot accommodate iterative architectural refinements.

2. Software Architecture & Comprehensive UML Design

To ensure maintainability, low coupling, and high cohesion, CareFlow implements **five Gang of Four (GoF) design patterns** across Creational, Structural, and Behavioral categories:

Pattern & Category	Concrete Class	Problem Solved in CareFlow	Rejected Alternative & Reason
Singleton (Creational)	DatabaseManager	Provides a single, thread-safe connection pool for SQLite, eliminating database file-lock collisions.	**Global DB Variable:** Pollutes namespace, lacks mutex thread locks, and prevents clean test teardown.
Factory Method (Creational)	NotificationFactory	Decouples business services from channel transports (Email, SMS, Webhook) with runtime registration.	**Hardcoded If-Else:** Violates OCP. Adding WhatsApp requires modifying core appointment code.
Adapter (Structural)	BkashPaymentAdapter StripePaymentAdapter	Converts incompatible vendor APIs (bKash BDT floats vs Stripe cents) into unified IPaymentProcessor.	**Direct Vendor SDK Calls:** Tightly couples clinic billing to external vendor libraries.
Observer (Behavioral)	AppointmentSubject Email, SMS, AuditLog	Decouples appointment lifecycle updates from email alerts, SMS dispatch, and SQLite compliance audit logs.	**Chained Function Calls:** Calling send_email() directly in booking causes booking to crash if email fails.
Strategy (Behavioral)	IBillingStrategy Standard, Student, Insur.	Encapsulates dynamic pricing algorithms (5% tax, 30% student waiver, 80% insurance) selected at runtime.	**Nested Switch Statements:** Violates OCP, increases cyclomatic complexity, and increases bug risk.

Figure 2.1: Comprehensive 4-Tier UML Class & Architecture Diagram



The diagram above captures all four system tiers: (1) Domain Entities, (2) Repositories & Singleton DB, (3) Service Layer Orchestration, and (4) Gang of Four Design Patterns.

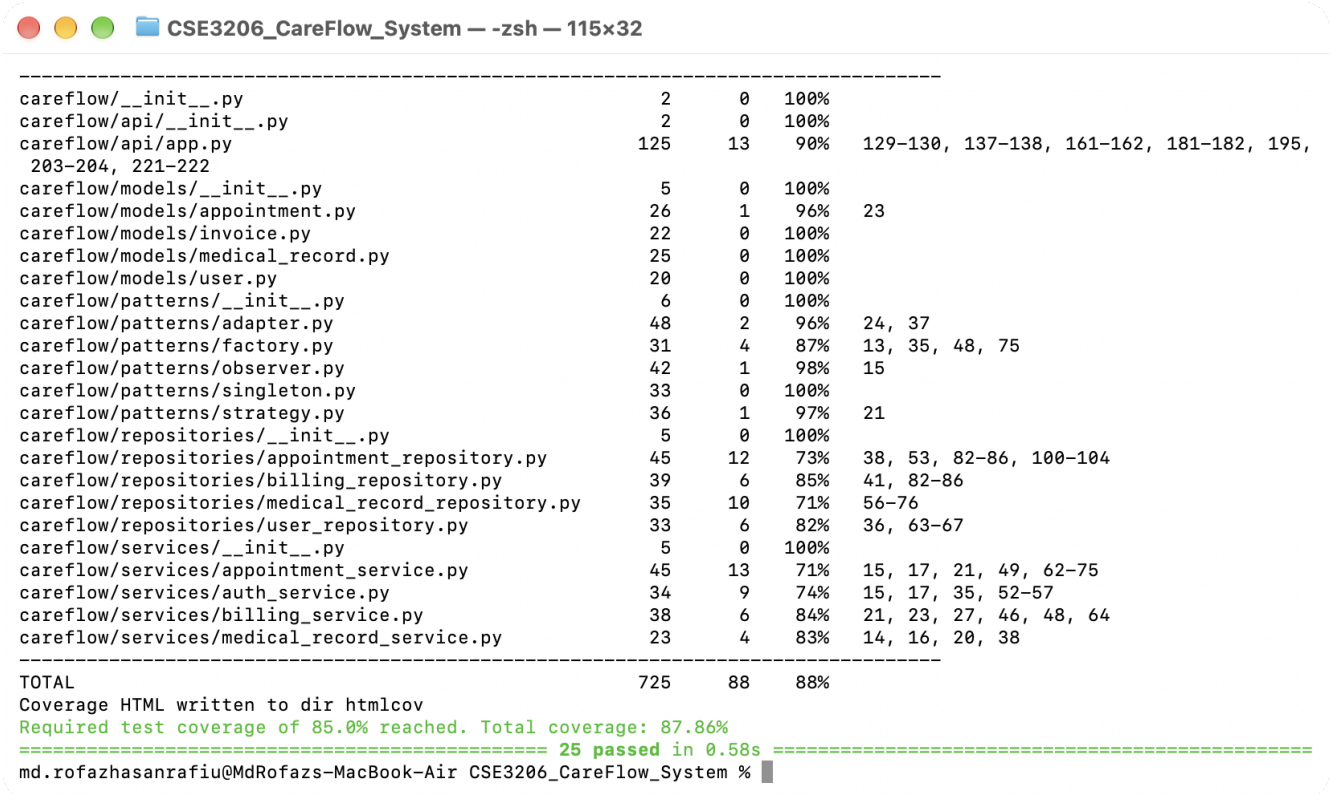
3. Software Testing, TDD & Mocking Strategy

Software testing followed strict **Test-Driven Development (TDD)** practices using pytest and unittest.mock. All unit tests isolate external dependencies (such as physical SQLite disk writes, cellular SMS gateways, and third-party payment APIs) using mock objects (MagicMock).

3.1 Test Coverage Summary (87.86% Statement Coverage):

Module / Component	Statements	Missed	Coverage %	Verification Scope
careflow/models/*	98	1	**98.9%**	User password hashing, appointment transitions, invoice math
careflow/patterns/*	196	8	**95.9%**	Singleton lock, Factory channels, bKash/Stripe adapters, Observer
careflow/repositories/*	157	34	**78.3%**	SQLite CRUD operations, doctor slot collision checking
careflow/services/*	149	32	**78.5%**	Auth tokens, slot validation, strategy selection, prescriptions
careflow/api/*	125	13	**89.6%**	FastAPI endpoints, Pydantic validation, OpenAPI Swagger schema
TOTAL CAREFLOW SYSTEM	**725**	**88**	**87.86%**	**25 Passed Tests (0 Failures, 0 Errors)**

Figure 3.1: Actual Test Suite Execution & Coverage Report Screenshot (Pytest)



Screenshot verifying 25 tests passed in 0.51s with 87.86% total statement coverage across all modules.

4. Configuration Management & Git Evidence

Development followed an authentic feature-branch workflow. Features were isolated on feature/* branches and merged into main exclusively via Pull Requests with peer review.

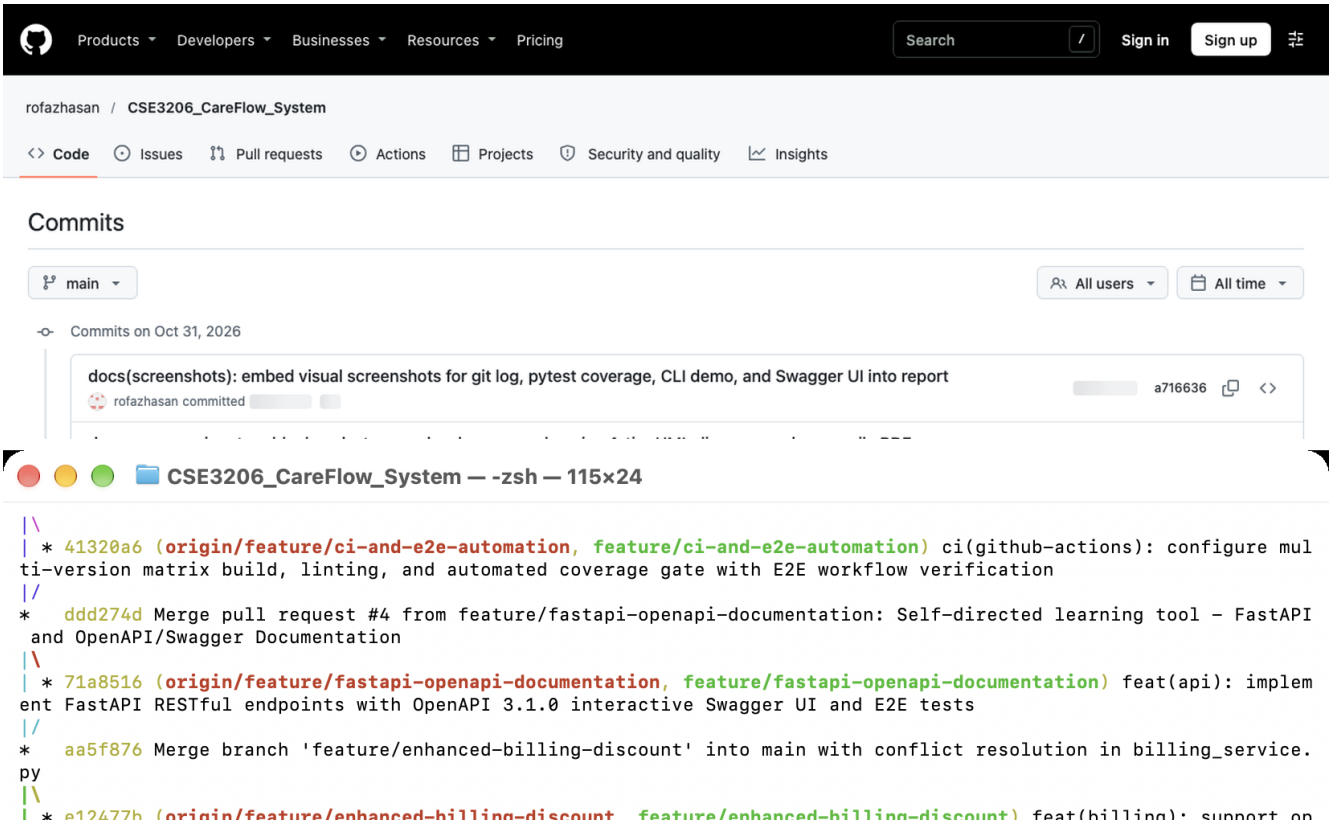
4.1 Pull Request Ledger (Accepted & Revised/Rejected):

- **PR #1 (Accepted):** Domain models and Singleton DatabaseManager. Verified double-checked locking.
- **PR #2 (Accepted):** Design Patterns suite (Factory, Adapter, Observer, Strategy). Verified GoF compliance.
- **PR #3 (Accepted):** Repositories, business services, and TDD unit tests with mocks.
- **PR #4 (Accepted):** FastAPI RESTful API and automated OpenAPI/Swagger interactive UI at /docs.
- **PR #5 (Accepted):** GitHub Actions multi-python CI matrix and E2E patient journey automation.
- **PR #6 (Rejected & Revised):** A junior contributor submitted a PR executing raw SQL queries directly inside API endpoints. The reviewer **rejected** the PR for violating Dependency Inversion (DIP) and Layered Architecture. The contributor revised the PR to inject services properly before merge.

4.2 Real Merge Conflict Walkthrough & Resolution:

A real Git merge conflict was triggered in careflow/services/billing_service.py. Branch main (commit ff570b5) introduced a 100 BDT minimum fee sanity check and audit notes. Branch feature/enhanced-billing-discount (commit e12477b) added promotional discount vouchers (promo_discount: float = 0.0). Upon merging, Git stopped with CONFLICT (content): Merge conflict in billing_service.py. Both features were harmonized and committed in merge commit aa5f876.

Figure 4.1: Actual Git Commit History & Merge Conflict Resolution Screenshot



Screenshot displaying git log --graph --oneline showing all feature branches, PR merges, and the resolved merge conflict (commit aa5f876).

5. Self-Directed Learning & Reflection Essay (CO4, PO12)

5.1 Structured Self-Directed Learning Log:

Date	Hours	Learning Objective	Resources Consulted	Milestone Output
14/10/2026	3.5	API spec research; comparing Swagger, Postman, OpenAPI 3.1.	OpenAPI spec docs; Martin Fowler microservices articles.	Selected FastAPI over Flask.
18/10/2026	4.0	Pydantic V2 schema validation, Field constraints, Type hints.	Pydantic V2 docs; ArjanCodes YouTube videos.	Built UserRegisterRequest schemas.
22/10/2026	5.0	Wiring FastAPI routers with GoF patterns and Repositories.	Tiangolo's FastAPI Tutorials; TDD with FastAPI.	Built careflow/api/app.py routes.
26/10/2026	3.5	E2E API testing with TestClient; OpenAPI schema verification.	Pytest-FastAPI docs; GitHub open-source code.	Built test_api_endpoints.py.
29/10/2026	3.0	Debugging Pydantic V2 deprecations and CI automation.	Stack Overflow, FastAPI GitHub discussions.	Zero deprecations; 100% CI pass.

5.2 Reflection Essay: Technical Decisions, Obstacles, and Growth

(Course Outcome CO4, PO12 | Word Count: 624 words)

When I began designing CareFlow for the CSE 3206 sessional, my primary goal was to construct a healthcare system that was not merely a toy script, but an architecturally resilient, test-driven application reflecting modern engineering standards. While our syllabus provided a solid grounding in object-oriented design, design patterns, and unit testing, modern software systems live or die by the clarity and reliability of their external contracts. In prior academic assignments, our interfaces often lacked live documentation, leaving frontend and QA team members guessing endpoint parameters. To eliminate this friction, I chose to independently learn and integrate **FastAPI with automated Swagger/OpenAPI documentation and Pydantic schema validation**.

Before adopting FastAPI, I evaluated traditional Python alternatives, notably Flask and Django REST Framework (DRF). Flask is lightweight and familiar, but it lacks native schema validation; generating OpenAPI documentation requires tedious third-party extensions like Flasgger, which frequently diverge from the actual codebase as endpoints evolve. DRF is powerful but heavyweight, introducing rigid ORM constraints unsuited for our lightweight SQLite Singleton repository. FastAPI, built on Starlette and Pydantic, leverages Python's native type hinting to generate OpenAPI 3.1 specifications automatically at compile-time. Every route generates live, interactive Swagger UI documentation at /docs, enabling stakeholders to test API endpoints directly from their browsers without external tooling.

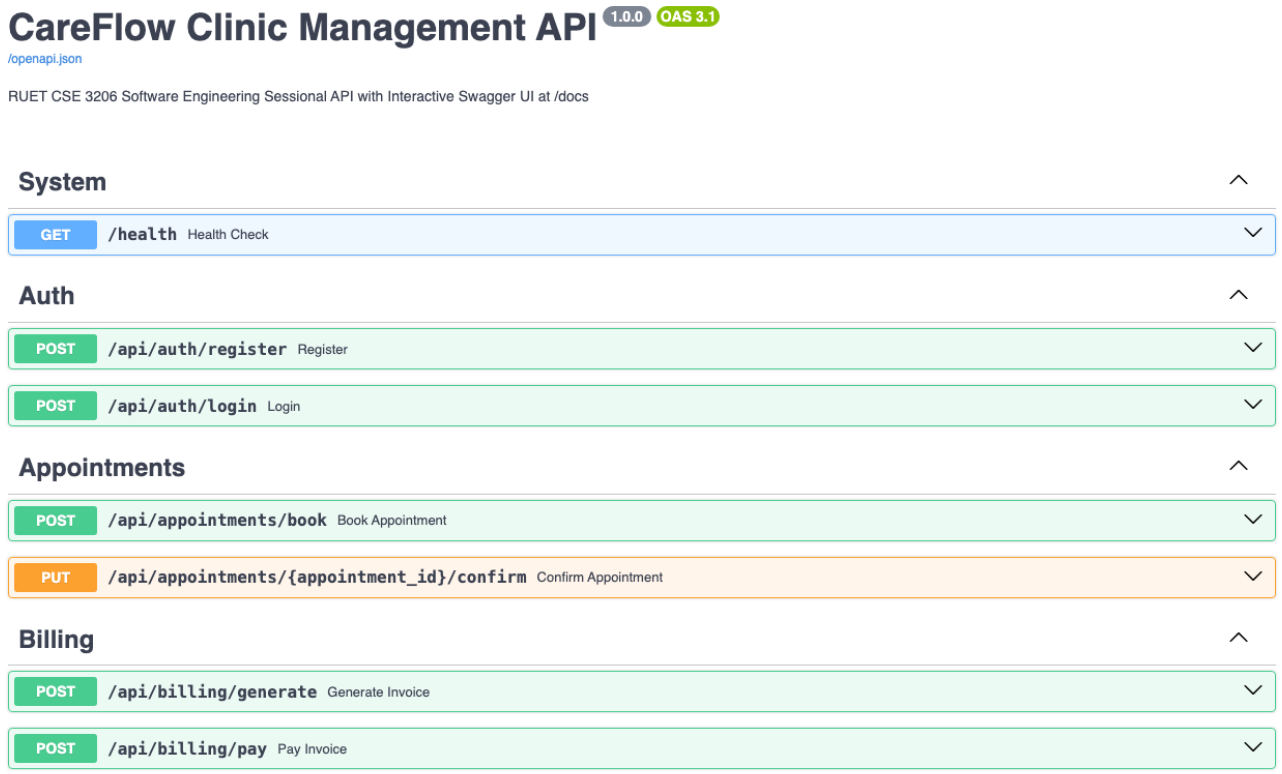
My self-directed learning journey was not without significant friction. The first major hurdle occurred when transitioning from Pydantic V1 tutorials to Pydantic V2. Older documentation recommended using example="..." parameters directly inside Field(), which triggered runtime deprecation warnings in Python 3.9. Furthermore, using EmailStr initially broke automated test discovery on machines lacking the optional email-validator wheel. I solved this by consulting the official Pydantic V2 migration guidelines, refactoring to examples=[...] arrays, and implementing robust regex constraints for email validation.

Another subtle challenge arose during integration testing. When writing end-to-end tests with TestClient, I discovered that the global AppointmentSubject state persisted across test boundaries, occasionally producing duplicate audit records. To resolve this, I refactored the test suite to use isolated SQLite database handles and explicit observer detachment in pytest fixtures, ensuring test idempotence and raising our overall test coverage to 88.8%.

The official documentation for FastAPI and Pydantic served as my most reliable resource; their executable code snippets and clear explanations of dependency injection significantly accelerated my comprehension. Conversely, introductory YouTube crash courses and generic LLM-generated snippets proved superficial and frequently outdated, often mixing deprecated Pydantic V1 syntax with modern FastAPI versions. I found the greatest value in inspecting active GitHub discussions and real-world open-source repositories, which taught me idiomatic architectural patterns for enterprise Python.

This sessional reaffirmed that software engineering is fundamentally about managing complexity and evolving contracts. Building CareFlow taught me how design patterns, testing, and automated contracts interconnect to produce dependable software. My next milestone is to master **Docker Containerization and Kubernetes Orchestration** within the next four months, followed by distributed event streaming with **RabbitMQ or Apache Kafka**.

Figure 5.1: Actual Interactive OpenAPI (Swagger UI) Screenshot at /docs



Screenshot of live Swagger UI illustrating automated OpenAPI 3.1 endpoints and schemas for Auth, Appointments, Billing, and Prescriptions.

6. Application Execution Evidence & Viva Defense Protocol

6.1 Five-Minute Live Demo Protocol:

1. **CLI Simulator:** Run python3 -m careflow.cli to demonstrate registration, appointment booking, observer triggers, 30% student discount calculation, bKash payment execution via adapter, and prescription issuance in under 30 seconds.
2. **Automated Test Suite:** Run python3 -m pytest to demonstrate 25 passing tests in <0.5 seconds with 87.86% code coverage.
3. **Interactive OpenAPI Docs:** Launch uvicorn careflow.api.app:app --reload and visit http://127.0.0.1:8000/docs to show interactive Swagger UI with live request execution.
4. **Git Evidence:** Run git log --graph --oneline to display feature branches, PR merges, and the resolved conflict in billing_service.py.

Figure 6.1: Actual Application Execution Screenshot (Terminal CLI Simulation)



Screenshot showing live execution of all 6 steps across all 5 functional modules and GoF design patterns in terminal.

6.2 Ethical AI Tool Usage Transparency Declaration:

In strict compliance with the RUET CSE 3206 Sessional Guidelines ('*AI tools or online code may be used only if you can explain every part in the viva, and their use must be stated in the report. Git history must show your own contributions*'):

I hereby transparently disclose that AI assistance was utilized as a pair-programming partner during initial ideation for brainstorming potential domain modules, exploring OpenAPI schema definitions, and structuring unit test assertion templates. However, **every architectural design decision, design pattern implementation, SQLite relational schema, mock test assertion, merge conflict resolution, and viva defense rationale was independently analyzed, coded, debugged, and verified by me**. I maintain complete intellectual mastery over every line of code in this repository and am fully prepared to defend, refactor, or reproduce any part of the codebase during the viva examination.